Data Structures & Algorithms

GOLDEN GATE UNIVERSITY

Session 7 – Queues and Circular Queues (FIFO Processing)

Dr. Yatish Bathla,

Ph.D. (Information Technology), Obuda University, Budapest

Lecture goals

Explain the FIFO rule and where it appears in real systems

Use the queue operations: insert (enqueue), remove (dequeue) and peek

See why a simple array queue wastes space, and how a circular queue fixes it

Implement a circular queue in Java and trace the wraparound

Compare array, circular and linked queues by cost

Recap: stack versus queue

Aspect	Stack (Session 6)	Queue (today)
Rule	LIFO: last in, first out	FIFO: first in, first out
Insert at	Top	Rear
Remove from	Top	Front
Operation names	push, pop, peek	insert/enqueue, remove/dequeue, peek
Everyday example	Stack of plates, undo	Ticket line, print jobs

Lecture plan and timing

Part	Topic	Minutes
Introduction	Slides 1 to 4	9
1 The queue idea	FIFO, ADT, picture (slides 5 to 7)	9
2 Array queue	Simple array queue and its limit (slides 8 and 9)	6
3 Circular queue	Wraparound, code, trace, full or empty (slides 10 to 14)	16
4 Efficiency	Cost comparison and linked queue (slides 15 and 16)	6
5 Java and FIFO use	Queue interface, print queue (slides 17 and 18)	6
6 Practice and recap	Quick check, tasks, summary (slides 19 and 20)	8

Reading: Lafore Ch.4, Queues (pp.132-143) | Goodrich Sec. 6.2, Queues (pp.238-248)

FIFO in the real world

Printing

Print spooler

Jobs are printed in the order they were sent. The oldest job goes first.

Waiting

Ticket line

Support tickets and counters: first come, first served. No one jumps the line.

Buffers

Keyboard, network

Data is read in the same order it was written, so nothing is reordered.

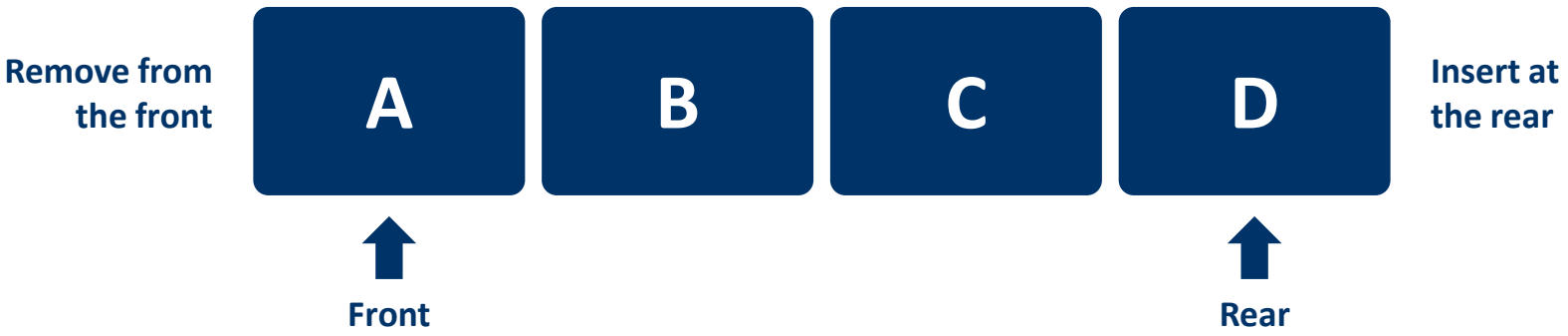
FIFO processing = the item that has waited longest is served next

The queue ADT

Lafore	Goodrich	What it does	Cost
insert(x)	enqueue(e)	Add an item at the rear	O(1)
remove()	dequeue()	Remove and return the front item	O(1)
peek()	first()	Read the front item, do not remove it	O(1)
isEmpty()	isEmpty()	True when there are no items	O(1)
size()	size()	Number of items in the queue	O(1)

Only the front can be removed and only the rear can be added to

A queue in pictures



enqueue(E): E joins behind D, the rear is now E

dequeue(): A leaves first and is returned, B becomes the front

peek(): look at A without removing it

Array queue: front and rear move up

```
public class ArrayQueue {
    private int[] data;
    private int front = 0;   // next item to remove
    private int rear = -1;   // last item inserted

    public ArrayQueue(int max) {
        data = new int[max];
    }

    public void insert(int x) {
        data[++rear] = x;    // rear moves up
    }

    public int remove() {
        return data[front++]; // front moves up
    }
}
```

Idea

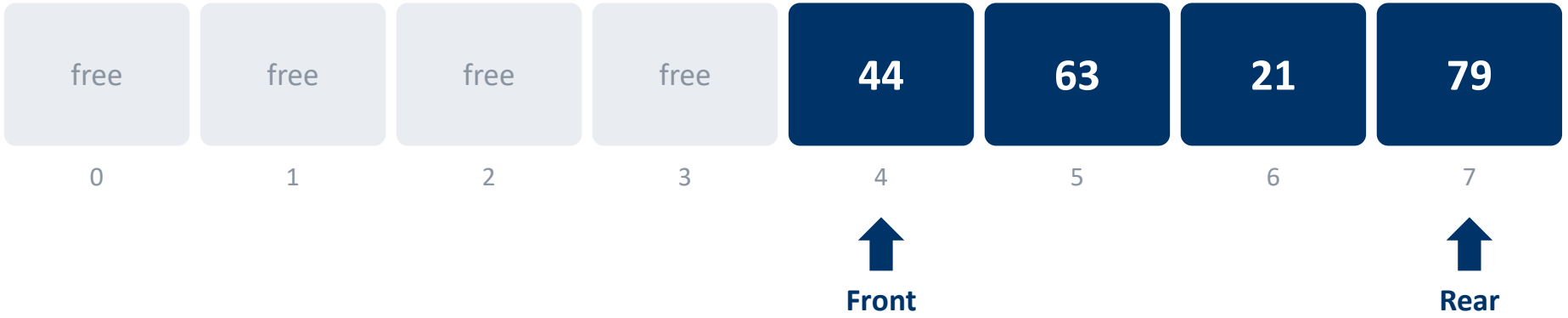
insert moves rear up by one

remove moves front up; no item is shifted

Each operation is O(1)

Missing: checks. What if rear reaches the end?

The problem: rear reaches the end



Rear is at the last index, so insert() cannot move it further.

Indexes 0 to 3 are free, yet the queue looks full.

Fix: shifting items costs O(n) per remove, so we wrap around instead (next slide).

Wraparound: the circular queue



Wraparound

Treat the array as a ring: after the last index comes index 0

next = (index + 1) % capacity

Both front and rear wrap, so nothing is ever shifted

Items may sit in two pieces of the array

Circular queue in Java: fields and insert

```
public class CircularQueue {
    private int[] data;
    private int front = 0;   // index of oldest item
    private int rear = -1;   // index of newest item
    private int nItems = 0;  // current count

    public CircularQueue(int max) {
        data = new int[max];
    }

    public void insert(int x) {
        if (isFull())
            throw new RuntimeException("full");
        rear = (rear + 1) % data.length;  // wrap
        data[rear] = x;
        nItems++;
    }
```

Read it

front, rear and nItems describe the queue

Check isFull() before changing anything

% data.length turns max index + 1 into 0

Lafore's version: if (rear == max - 1) rear = -1;

Circular queue in Java: remove and helpers

```
    public int remove() {
        if (isEmpty())
            throw new RuntimeException("empty");
        int item = data[front];
        front = (front + 1) % data.length; // wrap
        nItems--;
        return item;
    }

    public int peek() { return data[front]; }
    public boolean isEmpty() { return nItems == 0; }
    public boolean isFull() {
        return nItems == data.length;
    }
    public int size() { return nItems; }
}
```

Read it

remove reads first, then moves front

front wraps just like rear does

peek() and size() change nothing

Add a check to peek() as a task

Trace: watching rear wrap around

Step	Operation	front	rear	nItems	Array (index 0 to 4)
1	new CircularQueue(5)	0	-1	0	_ _ _ _ _
2	insert 10, 20, 30, 40	0	3	4	10 20 30 40 _
3	remove three times	3	3	1	_ _ _ 40 _
4	insert 50	3	4	2	_ _ _ 40 50
5	insert 60 (rear wraps)	3	0	3	60 _ _ 40 50
6	insert 70, 80	3	2	5	60 70 80 40 50

Removing all five items gives 40 50 60 70 80: still first in, first out

Full or empty? The same-looking state

Problem

front and rear look alike

Empty and full both give (rear + 1) % n == front. The indexes alone cannot decide.

Fix 1

Keep a counter

nItems is 0 when empty and n when full. Our code, Lafore and Goodrich use a count.

Fix 2

Leave one cell free

Capacity n - 1 usable cells. Full means (rear + 2) % n == front. No counter needed.

Both fixes keep every operation O(1)

Efficiency of queues

	Shifting array	Circular array	Linked queue
insert (enqueue)	O(1)	O(1)	O(1)
remove (dequeue)	O(n): shift all items	O(1)	O(1)
peek	O(1)	O(1)	O(1)
Capacity	Fixed	Fixed (resize costs O(n))	Grows as needed
Extra memory	None	None	One reference per item

Use a circular array when the size is bounded; a linked queue when it is not

A queue on a linked list

```
public class LinkedQueue {
    private Node head, tail;   // head = front

    public void enqueue(int x) {
        Node n = new Node(x);
        if (tail == null) head = n;   // was empty
        else tail.next = n;
        tail = n;
    }

    public int dequeue() {
        if (head == null)
            throw new RuntimeException("empty");
        int x = head.data;
        head = head.next;
        if (head == null) tail = null;  // now empty
        return x;
    }
}
```

Why two references

Remove at head: O(1)

Insert at tail: O(1) because we keep tail

Empty queue: head and tail are both null

Node class is the one from Lab 6

Java's built-in Queue

```
import java.util.ArrayDeque;
import java.util.Queue;

Queue<String> q = new ArrayDeque<>();
q.offer("A");                  // enqueue at rear
q.offer("B");
q.offer("C");
System.out.println(q.peek());  // A  (front)
System.out.println(q.poll());  // A  (removed)
System.out.println(q.poll());  // B
System.out.println(q.size());  // 1
System.out.println(q.poll());  // C
System.out.println(q.poll());  // null: queue empty
```

Notes

offer, poll, peek: return null or false when empty or full

add, remove, element: throw an exception instead

ArrayDeque is a resizable circular array

FIFO in action: a print queue

```
Queue<String> jobs = new ArrayDeque<>();
jobs.offer("report.pdf");
jobs.offer("photo.png");
jobs.offer("notes.txt");

int clock = 0;
while (!jobs.isEmpty()) {
    String job = jobs.poll();     // oldest first
    clock += 2;                   // 2 seconds per job
    System.out.println(clock + "s: printed " + job);
}
// 2s: printed report.pdf
// 4s: printed photo.png
// 6s: printed notes.txt
```

What to notice

First come, first served: fair by design

The 3rd job waits 4 seconds

Same loop shape appears in BFS and schedulers

Try: add a job inside the loop

Quick check and practice

Quick check

Insert A, B, C, then remove once. What does peek() return?

Capacity 5, front = 3, nItems = 4. Where is rear?

Why does a circular queue need nItems or a spare cell?

Your task

Type CircularQueue from slides 11 and 12

Run the trace from slide 13 and match every row

Add display(): print items from front to rear

Extension: when full, double the capacity and copy in queue order

Lecture recap and reflection

A queue is FIFO: insert at the rear, remove from the front

A plain array queue runs out of room at the end while cells are still free

A circular queue wraps front and rear with % capacity: every operation is O(1)

Track nItems (or leave one cell free) to tell full from empty

Next: recursion and divide-and-conquer, breaking a problem into smaller ones